

Part C: Databases and Data Debugging

C1: Average nightly revenue per property type (MySQL)

Full query with comments: c1_nightly_revenue_by_property_type.sql

```
SELECT
    property_type,
    currency,
    COUNT(*) AS completed_bookings,
    SUM(DATEDIFF(checkout_date, checkin_date)) AS total_nights,
    ROUND(AVG(total_amount / DATEDIFF(checkout_date, checkin_date)), 2) AS avg_nightly_revenue,
    ROUND(SUM(total_amount) / SUM(DATEDIFF(checkout_date, checkin_date)), 2) AS
weighted_nightly_revenue
FROM bookings
WHERE status = 'completed'
    AND created_at >= NOW() - INTERVAL 90 DAY
    AND checkout_date > checkin_date
GROUP BY property_type, currency
ORDER BY property_type, currency;
```

Notes on the choices:

- Nightly revenue is calculated per booking and then averaged, which is how the question defines it. I also return total revenue divided by total nights, because the two numbers drift apart when stay lengths vary, and whoever reads the report should know which one they're looking at.
- The result is split by currency. The table has both IDR and USD, and an average across the two means nothing. If there is an FX table, the .sql file has the converted version.
- `checkout_date > checkin_date` filters out zero-night, inverted and NULL-date rows, which would divide by zero or give negative rates. A second query in the file counts how many rows were dropped.
- "Last 90 days" is a rolling window on `created_at`. An index on `(status, created_at)` keeps this from scanning the whole table.

C2: Messages per intent per day, last 7 days (MongoDB)

Full script (pymongo): c2_messages_per_intent_per_day.py

```
tz = ZoneInfo("Asia/Jakarta")
start_of_today = datetime.combine(datetime.now(tz).date(), time.min, tzinfo=tz)
since = start_of_today - timedelta(days=6)

pipeline = [
    {"$match": {"created_at": {"$gte": since}}},
    {"$group": {
```

```

    "_id": {
      "day": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at", "timezone":
"Asia/Jakarta"}}},
      "intent": {"$ifNull": ["$intent", "unclassified"]},
    },
    "count": {"$sum": 1},
  }},
  {"$project": {"_id": 0, "day": "$_id.day", "intent": "$_id.intent", "count": 1}},
  {"$sort": {"day": 1, "intent": 1}},
]

rows = list(db.messages.aggregate(pipeline))

```

- Days are grouped in Jakarta time. Grouping in UTC would put a message sent at 06:00 WIB into the previous day.
- The start of the window (today plus the previous six days) is computed in Python before the query, so \$match is a simple range that can use an index on created_at.
- Messages with no intent are counted as unclassified instead of being lost from the totals.

C3: GMV down 40%, booking count normal

Reading the facts first

Volume is normal but value dropped, so the problem is the amount stored on bookings, not missing rows. Nothing was deployed to the booking service, and the one change that day is a new partner writing into the same table through an ingestion job. So the first thing I'd do is split everything by source.

One thing needs explaining early. A new partner should add bookings, yet the count is flat. That means either partner_x bookings replaced demand that used to come through another channel, or the partner rows overwrote existing rows instead of being added.

This query already separates most of the hypotheses below:

```

SET @onboarding = '2026-09-08';           -- last Tuesday

SELECT DATE(created_at) AS day, source, currency, status,
       COUNT(*) AS bookings,
       SUM(total_amount) AS amount,
       ROUND(AVG(total_amount / NULLIF(DATEDIFF(checkout_date, checkin_date), 0)), 2) AS avg_nightly
FROM bookings
WHERE created_at >= @onboarding - INTERVAL 14 DAY
GROUP BY day, source, currency, status
ORDER BY day, source, currency, status;

```

Hypothesis 1 (most likely): partner_x amounts are in the wrong unit or currency

This is the classic partner-feed bug. It takes one of these forms:

- USD amounts stored with currency = 'IDR' because of a default or a hardcoded mapping
- minor units instead of major units
- Indonesian thousand separators parsed as decimals ("1.500.000" becomes 1.5)
- net amounts after commission instead of gross

What it would look like: partner_x's average nightly rate is far below direct and ota for the same property type, while the other sources look normal. In the daily split, ota or direct counts would fall by roughly the number of partner_x bookings, which is why total volume didn't move.

How to confirm or kill it:

```
-- Same property type, same period: does partner_x price like the other channels?
SELECT source, property_type, currency, COUNT(*) AS n,
       ROUND(AVG(total_amount / NULLIF(DATEDIFF(checkout_date, checkin_date), 0)), 2) AS
avg_nightly,
       MIN(total_amount) AS min_amount, MAX(total_amount) AS max_amount
FROM bookings
WHERE created_at >= @onboarding
GROUP BY source, property_type, currency
ORDER BY property_type, source;

-- Same properties, partner_x vs other channels, before and after onboarding
SELECT property_id, source, created_at >= @onboarding AS after_onboarding,
       ROUND(AVG(total_amount / NULLIF(DATEDIFF(checkout_date, checkin_date), 0)), 2) AS avg_nightly
FROM bookings
WHERE property_id IN (SELECT DISTINCT property_id FROM bookings WHERE source = 'partner_x')
  AND created_at >= @onboarding - INTERVAL 30 DAY
GROUP BY property_id, source, after_onboarding;
```

Then compare about ten partner_x bookings with the partner's raw payload or invoice. If partner_x prices the same properties like the other channels and the stored amounts match the raw data, this hypothesis is dead.

Hypothesis 2: the ingestion job overwrote existing bookings

For example, partner booking ids collide with our id values and the loader uses REPLACE INTO or INSERT ... ON DUPLICATE KEY UPDATE, so our rows get replaced by partner rows with different amounts. This explains a flat count perfectly, because rows are replaced rather than added. It would also change numbers for days before the onboarding.

How to confirm or kill it:

```
-- 1. Have days before onboarding changed since they were reported? Compare with the archived daily reports.
SELECT DATE(created_at) AS day, COUNT(*) AS bookings, SUM(total_amount) AS amount
```

```

FROM bookings
WHERE created_at >= @onboarding - INTERVAL 14 DAY AND created_at < @onboarding
GROUP BY day;

-- 2. partner_x rows dated before the partner existed
SELECT COUNT(*) FROM bookings WHERE source = 'partner_x' AND created_at < @onboarding;

-- 3. Do partner ids fall inside our existing id range?
SELECT source, MIN(id), MAX(id), COUNT(*) FROM bookings GROUP BY source;

```

I'd also read the loader's insert statement and check the binlog for UPDATE/REPLACE on bookings from the ingestion user since Tuesday. If past days still match the archived reports, no partner_x row is older than the onboarding, and the loader does a plain INSERT, this one is dead.

Hypothesis 3: partner_x rows use values the GMV report filters out

For example status = 'COMPLETED' or 'confirmed', or currency = 'idr' or 'Rp'. The GMV report has WHERE status = 'completed' or an inner join to an FX table on currency, while the count report has neither, so partner bookings count as volume but add nothing to GMV.

How to confirm or kill it:

```

SELECT source, status, currency, COUNT(*)
FROM bookings
WHERE created_at >= @onboarding
GROUP BY source, status, currency;

```

Then compare the WHERE and JOIN of the GMV query with the count query, and run the GMV query for source = 'partner_x' only. If it returns nothing while the rows exist, that's the cause. If partner_x uses exactly 'completed' and 'IDR'/'USD', it isn't.

Why this order

- H1 fits every fact: same day, no deploy, value down but not volume, and a schema with currency sitting next to a brand-new source. It's also the most common mistake when onboarding a partner feed.
- H2 explains the flat count best, but it would have changed historical days, which takes five minutes to check.
- H3 would remove partner GMV, but unless the partner took traffic from another channel the count report would show more bookings, not the same.

Most likely root cause

The partner_x ingestion job writes total_amount in the wrong unit or currency, most likely USD (or net, or minor-unit) amounts stored as IDR. Since last Tuesday, the bookings that now come in through partner_x carry a fraction of their real value, so volume looks normal and GMV falls. The fix is the mapping in the job plus a backfill of the affected rows. A simple contract on the feed (allowed currencies, nightly rate within a normal range per property type, a daily reconciliation against partner totals) would have flagged it on the first day. Part E (d) describes that check.

C4 (bonus): slow daily count in ClickHouse

```
SELECT toDate(created_at) d, count() FROM events WHERE event_type = 'booking' GROUP BY d
```

Why it's slow: a MergeTree table's primary index is sparse and follows the ORDER BY key, which here is (id). It can only skip data when the filter is on id. This query filters on event_type and groups by created_at, so ClickHouse can't skip any granule and reads both columns for every row in every part. The cost grows with the table. There's also no date range in the query, so every run scans the full history, and without a date PARTITION BY there's nothing to prune.

What I'd change:

1. Rebuild the table with a sort key that matches how it's queried. ORDER BY can't be changed in place, so create a new table and swap:

```
CREATE TABLE events_v2
(
    id          UInt64,
    event_type  LowCardinality(String),
    created_at  DateTime
    -- other columns
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(created_at)
ORDER BY (event_type, toDate(created_at), id);

INSERT INTO events_v2 SELECT * FROM events;    -- in batches per month for a big table; pause
or dual-write ingestion
EXCHANGE TABLES events AND events_v2;
```

With event_type first, the filter only touches one key range. The date next means the group-by reads in order. Monthly partitions can be skipped entirely. If lookups by id still matter, a bloom filter skip index on id covers them.

2. Put a date range on the report, e.g. AND created_at >= today() - 90.
3. If the dashboard runs all the time, keep a daily rollup:

```
CREATE TABLE daily_event_counts (d Date, event_type LowCardinality(String), events UInt64)
ENGINE = SummingMergeTree ORDER BY (event_type, d);

CREATE MATERIALIZED VIEW daily_event_counts_mv TO daily_event_counts AS
SELECT toDate(created_at) AS d, event_type, count() AS events
FROM events GROUP BY d, event_type;
-- backfill older data once, up to the moment the view was created

SELECT d, sum(events) FROM daily_event_counts
WHERE event_type = 'booking' GROUP BY d ORDER BY d;
```

`sum( )` is needed because `SummingMergeTree` merges rows in the background, not immediately. A projection on the existing table is a lighter alternative if a second table isn't wanted.

Check before and after with `EXPLAIN indexes = 1` and `read_rows` in `system.query_log`.